



CALOWIN

*Crush your calories,
Conquer your goals*

SC2006 REP Group 38

Raphael Sim Yang Yue
Chen Zi Yan
Guan Xiaoxi
Lim Jun Jie
Tan Yi Da



TABLE OF CONTENTS

01

OVERVIEW

02

MAIN FUNCTIONS

03

SYSTEM DESIGN

04

OBJECT DESIGN

05

LIVE DEMO + TEST
CASE





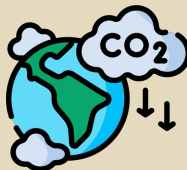
OVERVIEW

Calowin is a mobile application designed to promote environmental sustainability and support personal health by encouraging users to make eco friendly and active transportation choices. Unlike traditional navigation tools, Calowin does not only provide route planning or navigation functionalities. Instead, it emphasizes environmental and fitness tracking during a trip, calculating metrics such as carbon saved and calories burn through visiting different green areas in Singapore.

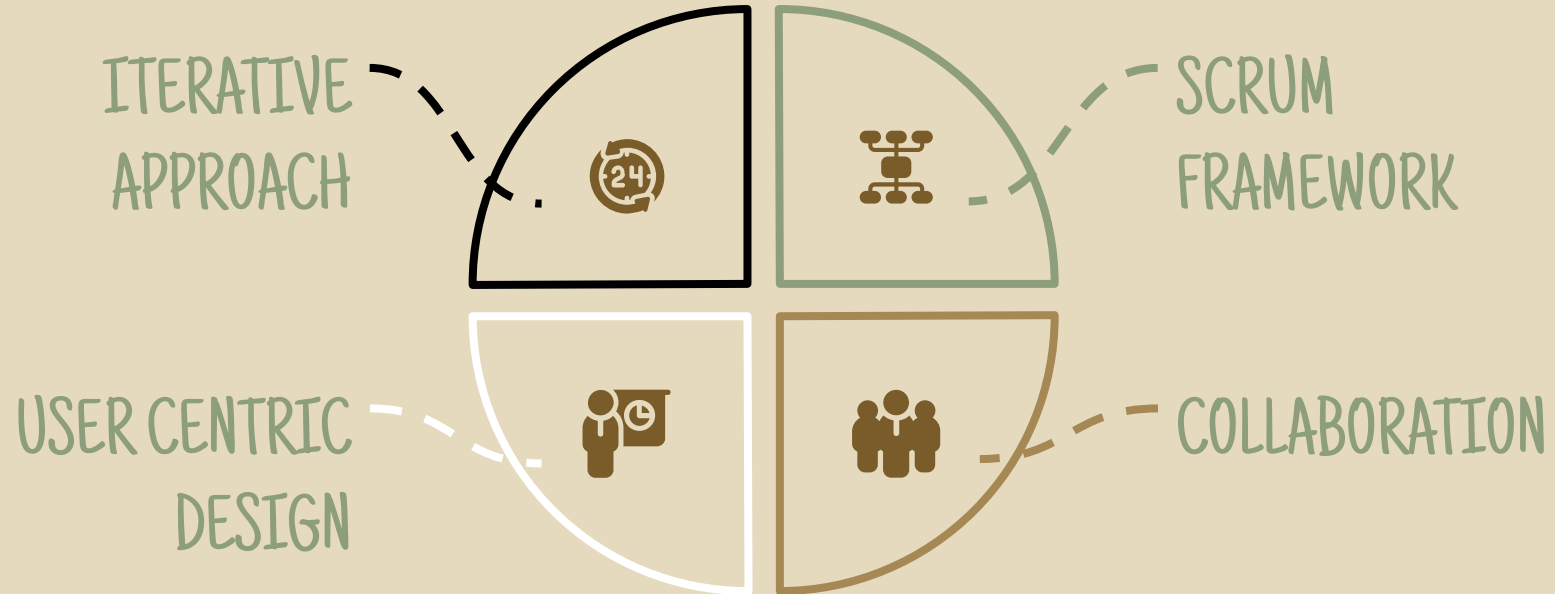
CALORIES BURNT



CARBON SAVED



AGILE DEVELOPMENT PROCESS





DEVELOPMENT FRAMEWORK



FRONT-END



BACK-END



MAIN FUNCTIONS

Accounts

- I. Account Sign Up
- II. Account Login
- III. Forget Password
- IV. OTP Email Service

Trips

- I. Google Map API Service
- II. Start Trip
- III. Cancel Trip
- IV. End Trip
- V. Calculate Trip Metrics

Friends

- I. Add Friends
- II. Manage Requests
- III. View Friend Profile

User Profile

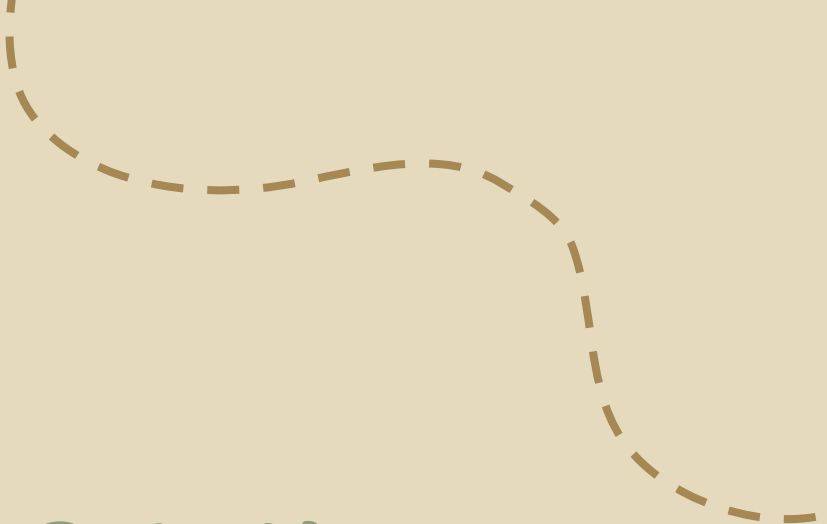
- I. View Profile
- II. Edit Profile
- III. Change Password
- IV. Account Deletion

Wellness Zones

- I. NParks API
- II. Live Weather API
- III. Optimise radius
- IV. View Wellness Zones
- V. Visit Wellness Zone

Leaderboard

- I. View Leaderboard
- II. Filter Leaderboard



SYSTEM DESIGN

MicroServices System Architecture

CalowinAccount

User Login

User Sign Up

<<extend>>

<<extend>>

<<include>>

<<include>>

<<extend>>

<<extend>>

<<include>>

<<include>>

Delete Account

Edit Profile

User Logout

View Profile (Self)

Verify Credentials

Change Password

Forgot Password

Email Service

WellnessZone

View Live Weather

Wellness Zone Recommendation

<<extend>>

<<include>>

Weather API

NParks API

Google Map API

CalowinFriends

Remove Friend

<<extend>>

View Profile (Friend)

Manage FriendList

<<include>>

<<include>>

View Profile (Non-Friend)

View Notifications

Manage Friend Request

<<include>>

<<include>>

Add Friend

<<extend>>

View Leaderboard

CalowinTrip

Select Trip

<<include>>

Start Trip

Cancel Trip

End Trip

<<include>>

Display Achievement Progress

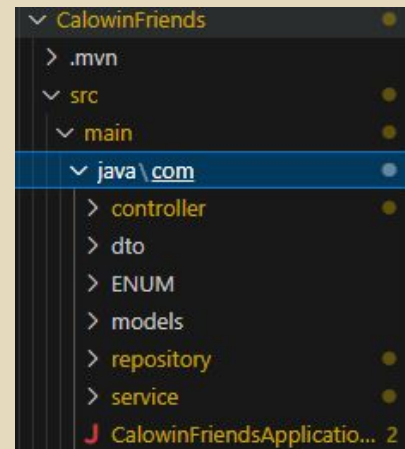
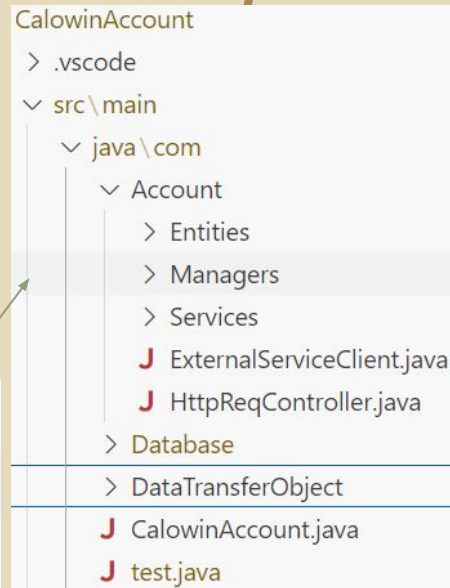
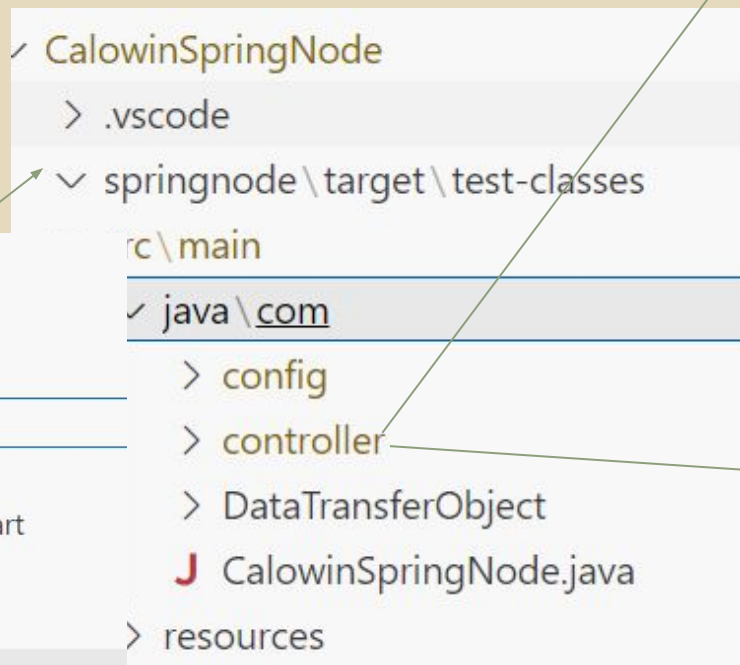
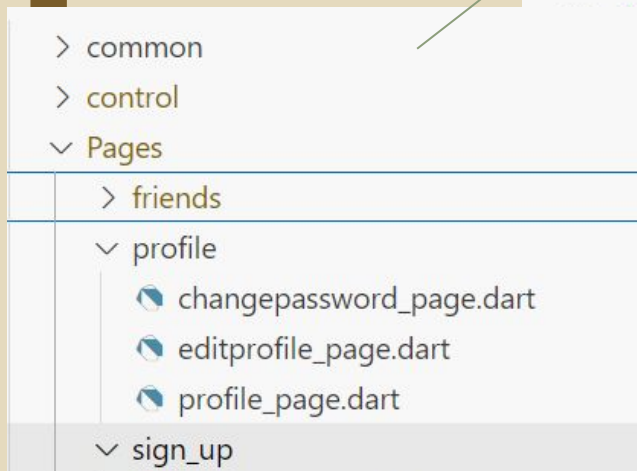
CalowinSpringnode

Frontend

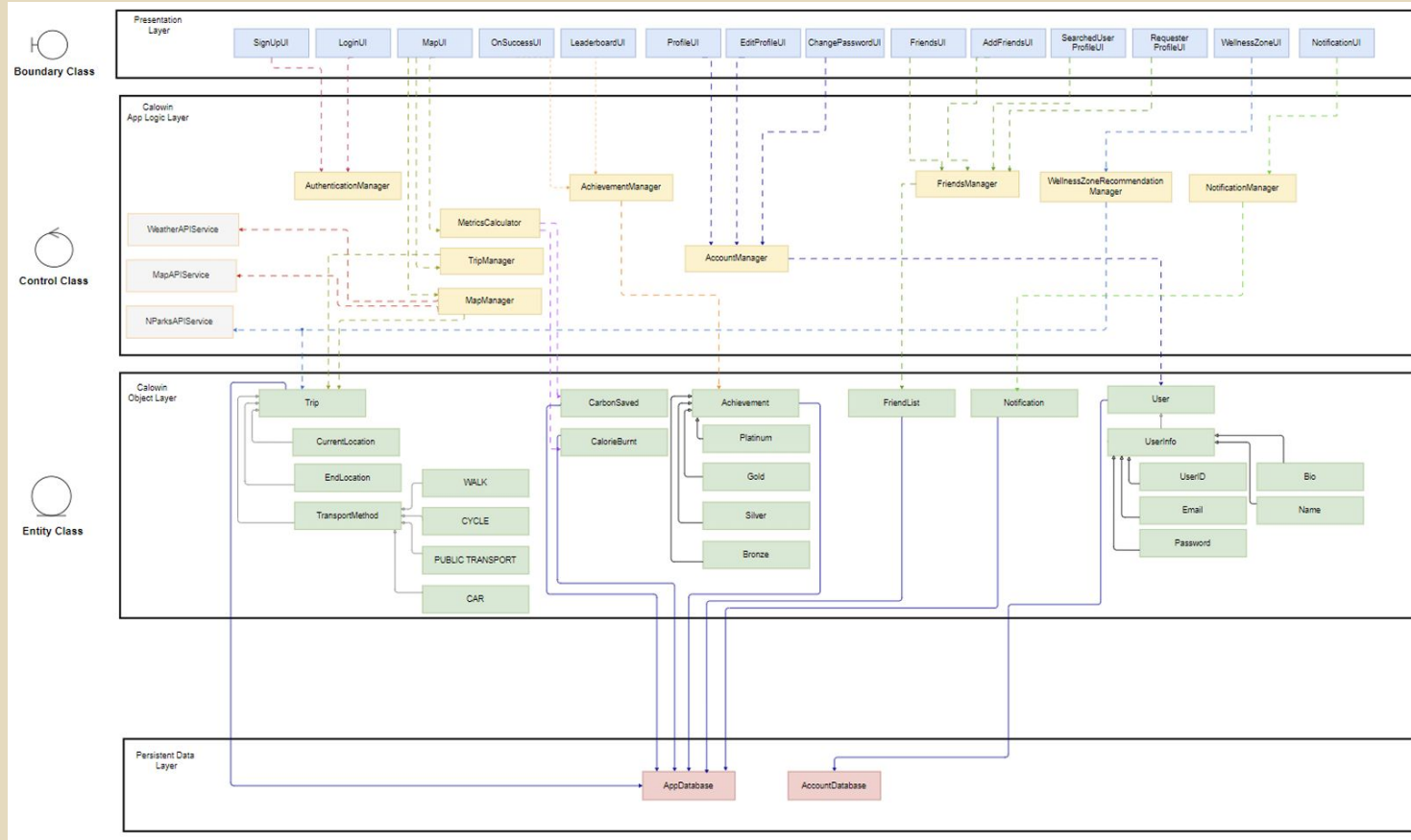


MICROSERVICES

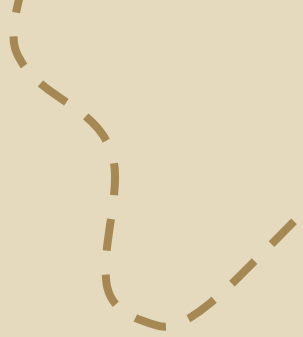
Examples



LAYERED DESIGN



OBJECT DESIGN



Design Principles

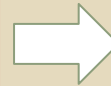
```
▼ DataTransferObject
J ChangePasswordDTO.java
J DeleteAccountDTO.java
J EditProfileDTO.java
J ForgotPasswordDTO.java
J LoginDTO.java
J LoginResponseDTO.java
J SendOtpDTO.java
J SignupDTO.java
J VerifyOtpDTO.java
J ViewProfileResponseDTO.java
```



Loose coupling Principle

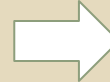
Separation of Concerns principle

```
▼ CalowinAccount
> .vscode
▼ src
  ▼ main
    ▼ java\com
      ▼ Account
        ▼ Entities
          J AchievementEntry.java
          J ActionType.java
          J EmailServiceProperties.java
          J FriendRelationshipEntry.java
          J FriendStatus.java
          J MyBeans.java
          J OTPEntry.java
          J ProfileEntity.java
          J TripsEntry.java
          J UserEntity.java
        ▼ Managers
          J AccountManagementService.
          J PasswordManagementServic.
          J ProfileManagementService.j..
        ▼ Services
          J EmailService.java
          J OTPService.java
          J PasswordSecurityService.java
          J ExternalServiceClient.java
          J HttpReqController.java
```



High Cohesion Principle

```
▼ Database
  ▼ CalowinDB
    J AchievementRepository.java
    J CalowinDBConfig.java
    J CalowinDBProperties.java
    J FriendRelationshipRepositor...
    J TripsRepository.java
    J UserInfoRepository.java
  ▼ CalowinSecureDB
    J CalowinSecureDBConfig.java
    J CalowinSecureDBProperties.j..
    J OTPRepository.java
    J SecureInfoDBRepository.java
    J DataSourceConfig.java
    J DBProperties.java
    J TransactionConfig.java 2
```



Open-Closed Principle



OBJECT DESIGN PATTERNS



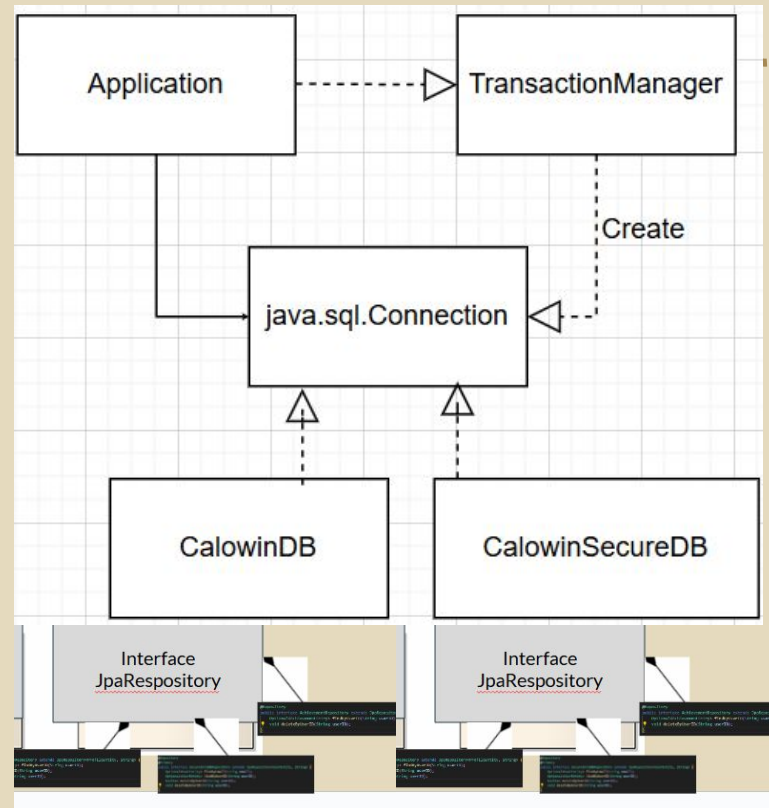
FACTORY DESIGN



STRATEGY DESIGN



DYNAMIC LOADING





```
public class CalowinSecureDBConfig {

    @Primary
    @Bean(name = "calowinSecureEntityManagerFactory")
    public LocalContainerEntityManagerFactoryBean calowinSecureEntityManagerFactory(
        EntityManagerFactoryBuilder builder, @Qualifier("calowinSecureDataSource") DataSource dataSource) {
        return builder
            .dataSource(dataSource)
            .packages(...packagesToScan("com.Account.Entities")) // Entity package for CalowinSecure
            .persistenceUnit("calowinSecureDB")
            .build();
    }

    @Primary
    @Bean(name = "calowinSecureDBTransactionManager")
    public PlatformTransactionManager calowinSecureDBTransactionManager(
        @Qualifier("calowinSecureEntityManagerFactory") EntityManagerFactory entityManagerFactory) {
        return new JpaTransactionManager(entityManagerFactory);
    }
}
```

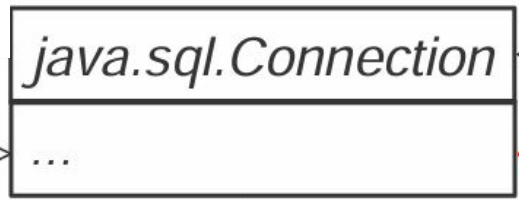
```
public class CalowinDBConfig {

    @Bean(name = "calowinDBEntityManagerFactory")
    public LocalContainerEntityManagerFactoryBean calowinDBEntityManagerFactory(
        EntityManagerFactoryBuilder builder, @Qualifier("calowinDBDataSource") DataSource dataSource) {
        return builder
            .dataSource(dataSource)
            .packages(...packagesToScan("com.Account.Entities")) // Entity package for CalowinDB
            .persistenceUnit("calowinDB")
            .build();
    }

    @Bean(name = "calowinDBTransactionManager")
    public PlatformTransactionManager calowinDBTransactionManager(
        @Qualifier("calowinDBEntityManagerFactory") EntityManagerFactory entityManagerFactory) {
        return new JpaTransactionManager(entityManagerFactory);
    }
}
```

```
@Configuration
@EnableTransactionManagement
public class TransactionConfig {

    @Bean(name = "chainedTransactionManager")
    public PlatformTransactionManager chainedTransactionManager(
        @Qualifier("calowinSecureDBTransactionManager") PlatformTransactionManager calowinSecureDBTransactionManager,
        @Qualifier("calowinDBTransactionManager") PlatformTransactionManager calowinDBTransactionManager) {
        return new ChainedTransactionManager(calowinSecureDBTransactionManager, calowinDBTransactionManager);
    }
}
```



create

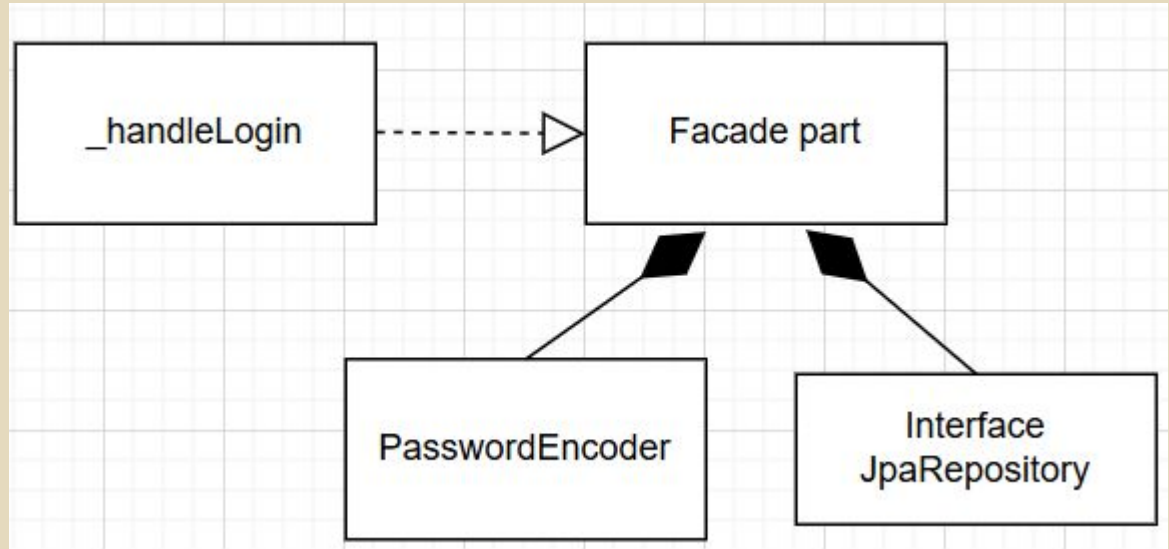
```
@Bean(name = "calowinDBDataSource")
public DataSource calowinDBDataSource() {
    AtomikosDataSourceBean ds = new AtomikosDataSourceBean();
    ds.setUniqueResourceName(resourceName: "calowinDBDataSource");
    ds.setXaDataSourceClassName(calowinDBProperties.getXaDataSourceClassName());
    ds.setXaProperties(calowinDBProperties.toXaProperties());
    ds.setMinPoolSize(calowinDBProperties.getMinPoolSize());
    ds.setMaxPoolSize(calowinDBProperties.getMaxPoolSize());
    return ds;
}
```

```
@Bean(name = "calowinSecureDBDataSource")
public DataSource calowinSecureDBDataSource() {
    AtomikosDataSourceBean ds = new AtomikosDataSourceBean();
    ds.setUniqueResourceName(resourceName: "calowinSecureDBDataSource");
    ds.setXaDataSourceClassName(calowinSecureDBProperties.getXaDataSourceClassName());
    ds.setXaProperties(calowinSecureDBProperties.toXaProperties());
    ds.setMinPoolSize(calowinSecureDBProperties.getMinPoolSize());
    ds.setMaxPoolSize(calowinSecureDBProperties.getMaxPoolSize());
    return ds;
}
```



OBJECT DESIGN PATTERNS

Facade Design




```
class LoginpageState extends State<LoginPage> {
  final TextEditingController _inputPassword = TextEditingController();
  final TextEditingController _inputEmail = TextEditingController();
```

Front end Client

```
bool _wrongPassword = false;
bool _invalidEmail = false;

Future<void> _handleLogin(BuildContext context) async {
  setState(() {
    _wrongPassword = false;
    _invalidEmail = false;
  });
}
```

```
@PostMapping("/login")
public ResponseEntity<> login(@RequestBody LoginDTO loginDTO) {
  // Login logic (check username/password)
  try {
    LoginResponseDTO responseDTO = accountManagementService.login(loginDTO.getEmail(), loginDTO.getPassword());

    // Prepare response after successful login
    Map<String, Object> response = new HashMap<>();
    response.put(key:"message", value:"login successful");
    response.put(key:"UserObject", responseDTO);

    System.out.println(response);
    return ResponseEntity.ok(response);
  } catch (RuntimeException e) {
    // Return unauthorized error for invalid credentials
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(e.getMessage());
  } catch (Exception e) {
    // Handle other exceptions
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
      .body("Error during login: " + e.getMessage());
  }
}
```

Back end Client

Facade

```
public LoginResponseDTO login(String email, String encryptedPassword) throws Exception {
  String decryptedPassword = passwordSecurityService.decrypt(encryptedPassword, SECRET_KEY);

  UserEntity user = secureInfoRepository.findByEmail(email)
    .orElseThrow(() -> new RuntimeException(message:"Invalid email or password"));

  if (!passwordEncoder.matches(decryptedPassword, user.getPassword())) {
    throw new RuntimeException(message:"Invalid email or password");
  }

  ProfileEntity profile = userInfoRepository.findById(user.getUserID())
    .orElseThrow(() -> new Exception(message:"Failed to retrieve user data"));

  AchievementEntry achievement = achievementRepository.findById(user.getUserID())
    .orElseThrow(() -> new Exception(message:"Failed to retrieve achievement data"));

  return new LoginResponseDTO(user.getUserID(), user.getEmail(), profile.getName(), profile.getProfileImage(),
    achievement.getTotalCarbonSaved(), achievement.getTotalCaloriesConsumed());
}
```

Class 1

Class
PasswordEncoder

Class 2

Interface
JpaRepository

Strategy in a Facade

```
@Repository
public interface AchievementRepository extends JpaRepository {
  Optional<AchievementEntry> findById(String userID);
  void deleteById(String userID);
}
```

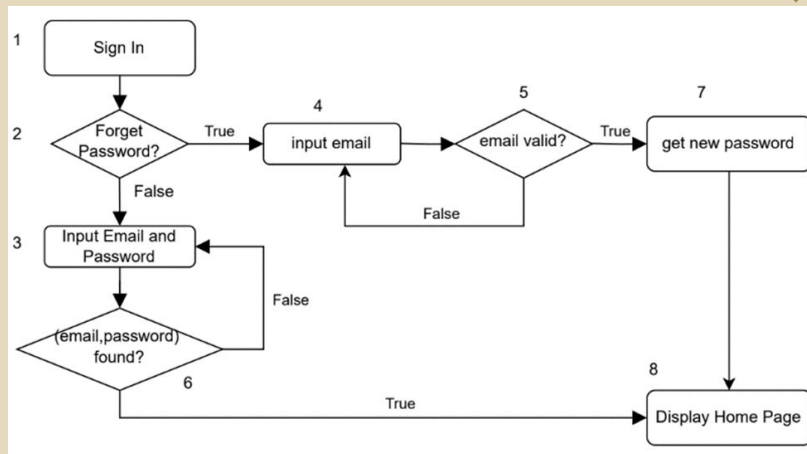
```
@Repository
public interface UserInfoRepository extends JpaRepository<ProfileEntity, String> {
  Optional<ProfileEntity> findById(String userID);
  boolean existById(String userID);
  void deleteById(String userID);
}
```

```
@Repository
@Primary
public interface SecureInfoRepository extends JpaRepository<UserEntity, String> {
  Optional<UserEntity> findById(String email);
  Optional<UserEntity> findById(String userID);
  boolean existById(String userID);
  void deleteById(String userID);
}
```

TEST CASES: BLACKBOX AND WHITEBOX TESTING

1. Account Registration

Test Scenario ID↵	RG-1↵		Test Case ID↵		RG-1A↵		
Test Case Description↵	Registration – Positive Test Case↵		Test Priority↵		High↵		
Pre-Requisite↵	Stable Internet Connection↵↵		Post-Requisite↵		NA↵		
Test Execution Steps:↵							
S.↵ No↵	Action↵	Inputs↵	Expected Output↵	Actual Output↵	Test Browser↵	Test Result↵	Test Comments↵
1↵	Launch application and click Sign Up↵	flutter run, Sign Up↵	Sign Up Page↵	Login Page↵	IOS Simulator↵	Pass↵	[Jun Jie 6/11/2024 7:56 PM]: Launch successful↵↵
2↵	Sign up with valid credentials and enter Valid OTP from Email↵	Valid Email Address↵↵ Password at least 8 characters, 1 uppercase, 1 lowercase and 1 special character.↵↵ Valid OTP Key↵	Move on to the User Information Page↵	OTP Page Loaded↵	IOS Simulator↵	Pass↵	[Jun Jie 6/11/2024 7:57 PM]: Credential Input Successful↵↵↵
3↵	Enter Name and Weight ↵	Name ↵↵ Numerical Number Weight↵	Move on to the User Information Page↵	Login Page Entered↵	IOS Simulator↵	Pass↵	[Jun Jie 6/11/2024 7:58 PM]: Signup Successful↵↵



Basis Set of Path

Path 1: 1,2,4,5,7,8 (forgot password with valid email)

Path 2: 1,2,3,6,8 (successful login)

Path 3: 1,2,3,6 (failed login attempt)

Path 4: 1,2,4,5,4,5,7,8 (forgot password with successful retry after entering invalid email)

HIGHLIGHTS

High cohesion and loose coupling contributes to high **maintainability**.

Microservices structure contributes to **scalability**.

Test cases-> **Testability**

New functionalities can be added over time without significantly changing the existing codebase->**Extensibility**

The use of object design patterns is evidence of reusable code components->**Reusability**



LIVE DEMO

